

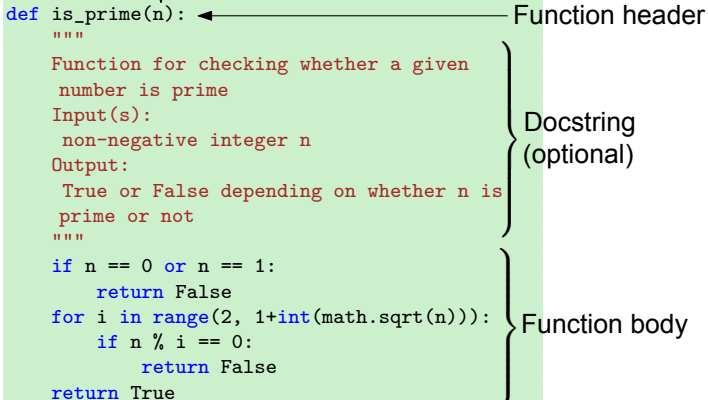
Python: functions

Intro. to computing

`https://mandarm.github.io/computing.html`

Functions I

Function parameters (zero or more, comma separated)



The diagram shows a Python function definition for `is_prime(n)`. An arrow points from the text 'Function parameters (zero or more, comma separated)' to the parameter `n` in the function signature. The function definition is enclosed in a green box. To the right of the box, two curly braces are used for annotation: one for the docstring and one for the function body.

```
def is_prime(n):  
    """  
    Function for checking whether a given  
    number is prime  
    Input(s):  
        non-negative integer n  
    Output:  
        True or False depending on whether n is  
        prime or not  
    """  
    if n == 0 or n == 1:  
        return False  
    for i in range(2, 1+int(math.sqrt(n))):  
        if n % i == 0:  
            return False  
    return True
```

Function header

Docstring (optional)

Function body

Functions II

- *Parameters* (or formal parameters): `n` in the above example
- *Arguments* (or actual parameters): value passed when the function is actually called
Example: `is_prime(4)`, `is_prime(5)` — 4 and 5 are arguments
- *return* statements
 - multiple occurrences permitted (as in example above)
 - may be followed by an optional expression (including `None`)
- *Function call signature*: number and type of arguments + return value (cf. $f : D_1 \times D_2 \times \dots \times D_k \rightarrow R$)

Naming convention

- Use lowercase letters
 - Use underscore (`_`) as word separator
- Example: `read_data()` preferred to `readData()`

Parameters / arguments

- Number of arguments: fixed vs. variable
- Type of arguments: required (compulsory) vs. optional
- Type of arguments: positional vs. keyword
- **Special markers:** *, /

Number of arguments

```
def multiply2(x, y): # 2 arguments
    return x * y

def multiply(*args): # could also use *numbers
    product = 1
    for x in args:
        product *= x
    return product

def multiply_flexible(initial, *numbers):
    product = initial
    for x in numbers:
        product *= x
    return product
```

- If parameter name is prefixed with *, it becomes a tuple containing *remaining* (0 or more)

Number of arguments

```
def multiply2(x, y): # 2 arguments
    return x * y

def multiply(*args): # could also use *numbers
    product = 1
    for x in args:
        product *= x
    return product

def multiply_flexible(initial, *numbers):
    product = initial
    for x in numbers:
        product *= x
    return product
```

What happens if you swap initial and *numbers

- If parameter name is prefixed with *, it becomes a tuple containing *remaining* (0 or more)

Positional vs. keyword arguments I

```
def multiply_flexible(*numbers, initial):  
    product = initial  
    for x in numbers:  
        product *= x  
    return product  
  
>>> multiply_flexible(2,3,4,10)           # carefully study error message  
>>> multiply_flexible(2,3,4,initial=10)  # 240
```

- **All** arguments following `*args` must be passed using `parameter_name=value` format.

Positional vs. keyword arguments II

- Can use `*` (with / without a parameter name) to force subsequent arguments to be passed as keyword arguments
 - `*` without a parameter name does not match anything

```
def example(x, y, *, a, b, c):  
    print("x =", x, "\ny =", y, "\na =", a, "\nb =", b, "\nc =", c)  
  
>>> example(1,2,3,4,5)           # carefully study error message  
>>> example(1,2,b=3,c=4,a=5)
```

Positional vs. keyword arguments III

Question: What does `example(y=2,x=1,b=3,c=4,a=5)` print?

- Parameters before `*` can be passed either by position, or as `keyword=value`.
 - all required arguments must be provided (cannot be omitted)
 - positional arguments must precede keyword-value arguments
 - any keyword must match a parameter name
 - multiple values should not be provided for any parameter

Questions:

- What does `example(b=3,c=4,a=5,y=2,x=1)` print?

Positional vs. keyword arguments IV

- Can use `/` to force *preceding* arguments to be passed as positional arguments

Required vs. optional arguments

Things to keep in mind

- Arguments fully evaluated left-to-right before executing function body